

# Cryptography

---

## 1. Introduction

Cryptography is the science and practice of securing information through mathematical algorithms to prevent unauthorized access, interpretation, or modification of data. More formally, it is the art or science encompassing the principles and methods of transforming an intelligible message into one that is unintelligible, and then retransforming that message back to its original form.

From a cybersecurity standpoint, cryptography is essential for protecting sensitive information stored, processed, and transmitted across networks. It ensures **confidentiality, integrity, authentication, and non-repudiation** — forming the backbone of numerous security protocols used in modern computing systems.

---

## 2. Basic Terminology

| Term                                         | Definition                                                                                                             |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Plaintext</b>                             | The original, intelligible message — understandable by sender, receiver, and anyone who gets access to it              |
| <b>Ciphertext</b>                            | The transformed, coded message                                                                                         |
| <b>Cipher</b>                                | An algorithm for transforming plaintext into ciphertext by substitution and/or transposition methods                   |
| <b>Key</b>                                   | Critical information used by the cipher, known only to sender and receiver                                             |
| <b>Encipher<br/>(encrypt /<br/>encode)</b>   | The process of converting plaintext to ciphertext using a cipher and a key                                             |
| <b>Decipher<br/>(decrypt /<br/>decode)</b>   | The process of converting ciphertext back into plaintext using a cipher and a key                                      |
| <b>Cryptography</b>                          | The study of encryption principles and methods                                                                         |
| <b>Cryptanalysis<br/>(code<br/>breaking)</b> | The study of principles and methods of transforming ciphertext back into plaintext <b>without</b> knowledge of the key |

| Term              | Definition                                                                                         |
|-------------------|----------------------------------------------------------------------------------------------------|
| <b>Cryptology</b> | The combined field of both cryptography and cryptanalysis                                          |
| <b>Code</b>       | An algorithm for transforming an intelligible message into an unintelligible one using a code-book |

## 2.1 The General Encryption Model

A source produces a plaintext message,  $X = [X_1, X_2, \dots, X_M]$ . A key  $K = [K_1, K_2, \dots, K_J]$  is generated and, if produced at the source, must be provided to the destination via a secure channel. With message  $X$  and key  $K$  as input, the encryption algorithm forms ciphertext  $Y$ :

$Y = E(K, X)$  (encryption)  
 $X = D(K, Y)$  (decryption – the intended receiver inverts the transformation)

An opponent observing  $Y$ , without access to  $K$  or  $X$ , may attempt to recover  $X$  (a plaintext estimate for this one message) or  $K$  (to be able to read *future* messages as well). It is generally assumed the opponent knows the encryption/decryption algorithms themselves — **security depends on the secrecy of the key, not the secrecy of the algorithm.**

## 3. Cryptanalysis

Cryptanalysis is the process of attempting to discover the plaintext  $X$ , the key  $K$ , or both, without authorized access. The strategy used depends on the nature of the encryption scheme and how much information is available to the cryptanalyst.

| Attack Type              | What the Cryptanalyst Has                                                                                                        |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Ciphertext-only</b>   | A copy of the ciphertext alone                                                                                                   |
| <b>Known-plaintext</b>   | A copy of the ciphertext and its corresponding plaintext                                                                         |
| <b>Chosen-plaintext</b>  | Temporary access to the encryption machine — can encrypt chosen plaintexts and study the resulting ciphertexts to deduce the key |
| <b>Chosen-ciphertext</b> | Temporary access to the decryption machine — can decrypt chosen strings and use the results to deduce the key                    |

## 4. Steganography

A plaintext message can be hidden in one of two ways: **steganography** conceals the *existence* of the message, whereas **cryptography** renders the message unintelligible to outsiders through transformation. Steganography involves hiding data within other data formats (text, images, audio, video) — unlike encryption, its focus is concealment of communication itself, not scrambling its content.

Historical techniques include:

- **Word/letter arrangement** — e.g., the first letter of each word in an innocuous message spells out the real message, or a chosen subset of words conveys the hidden content.
- **Character marking** — selected letters overwritten faintly in pencil, visible only at an angle under bright light.
- **Invisible ink** — substances that leave no visible trace until heat or a chemical is applied.
- **Pin punctures** — tiny punctures on selected letters, visible only when held to light.
- **Typewriter correction ribbon** — results visible only under strong light.

**Drawbacks:** requires a lot of overhead to hide relatively few bits of information, and once the system/method is discovered it becomes virtually worthless.

---

## 5. Security Services, Mechanisms, and Attacks

### 5.1 Security Services

| Service                | Ensures                                                                                                                                                         |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Confidentiality</b> | Information in a system and in transit is accessible only for reading by authorized parties (e.g., against printing, displaying, or other disclosure)           |
| <b>Authentication</b>  | The origin of a message/document is correctly identified, with assurance the identity is not false                                                              |
| <b>Integrity</b>       | Only authorized parties can modify system assets/transmitted information — covers writing, changing status, deleting, creating, and delaying/replaying messages |

| Service                | Ensures                                                                   |
|------------------------|---------------------------------------------------------------------------|
| <b>Non-repudiation</b> | Neither sender nor receiver can deny having transmitted a message         |
| <b>Access control</b>  | Access to information resources is controlled by or for the target system |
| <b>Availability</b>    | System assets are available to authorized parties when needed             |

## 5.2 Security Mechanisms

Encryption (or encryption-like transformation) is the most common means of providing security. The principal mechanisms are:

1. **Encipherment**
2. **Digital Signature**
3. **Access Control**

## 5.3 Security Attacks — Four General Categories

| Category            | Description                                                                                                                                           | Attacks         |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| <b>Interruption</b> | An asset is destroyed or becomes unavailable/unusable                                                                                                 | Availability    |
| <b>Interception</b> | An unauthorized party gains access to an asset (person, program, or computer — e.g. wiretapping, illicit file copying)                                | Confidentiality |
| <b>Modification</b> | An unauthorized party gains access <i>and</i> tampers with an asset (e.g., altering a data file or a program, modifying transmitted message contents) | Integrity       |
| <b>Fabrication</b>  | An unauthorized party inserts counterfeit objects into the system (e.g., a spurious message, added file records)                                      | Authenticity    |

## 5.4 Cryptographic Attacks — Passive vs. Active

**Passive attacks** are in the nature of eavesdropping on or monitoring transmissions, aiming to obtain information being transmitted. They are very difficult to *detect* (no data is altered) but feasible to *prevent*.

- **Release of message contents** — preventing an opponent from learning the contents of a sensitive transmission (call, email, file).

- **Traffic analysis** — even with encryption in place, an opponent may observe message patterns: location/identity of communicating hosts, frequency and length of exchanged messages, useful for guessing the nature of the communication.

**Active attacks** involve modification of the data stream or creation of a false stream. Difficult to prevent absolutely (would require constant physical protection of every communication path); the realistic goal is to *detect* them and *recover* from resulting disruption.

- **Masquerade** — one entity pretends to be a different entity.
  - **Replay** — passive capture of a data unit followed by its retransmission to produce an unauthorized effect.
  - **Modification of messages** — some portion of a legitimate message is altered, delayed, or reordered to produce an unauthorized effect.
  - **Denial of service** — prevents or inhibits normal use/management of communication facilities; can extend to disabling or overloading an entire network.
- 

## 6. A Model for Network Security

A message is transferred from one party to another across some sort of internetwork. Both parties are principals in the transaction and must cooperate for the exchange to take place. A logical information channel is established via a defined route and cooperative use of communication protocols (e.g., TCP/IP).

Using this model requires:

- designing a suitable algorithm for the security transformation
- generating the secret information (keys) used by the algorithm
- developing methods to distribute and share the secret information
- specifying a protocol enabling the principals to use the transformation and secret information for a security service

A **trusted third party** (arbiter/distributor of secret information) may be needed to achieve secure transmission, mediating between sender and recipient over the information channel while an opponent attempts to compromise it.

### 6.1 Model for Network Access Security

This complementary model addresses unwanted access — from human hackers to malicious software (viruses, worms) — via an access channel guarded by a **gatekeeper function** to an information system holding computing resources, data, processes, software, and internal security controls.

Using this model requires:

- selecting appropriate gatekeeper functions to identify users
- implementing security controls to ensure only authorized users access designated information/resources

Trusted computer systems can be used to implement this model.

## 7. Conventional (Symmetric-Key) Encryption

Also referred to as **conventional / private-key / single-key** encryption. Sender and recipient share a common key. All classical encryption algorithms were private-key — the only type prior to the invention of public-key cryptography in 1970s.

**Two requirements for secure use of symmetric encryption:**

1. A strong encryption algorithm
2. A secret key known only to sender and receiver

|             |                                                             |
|-------------|-------------------------------------------------------------|
| $Y = EK(X)$ | assume the encryption algorithm itself is known/public      |
| $X = DK(Y)$ | implies the need for a secure channel to distribute the key |

Symmetric-key algorithms use the same cryptographic key for both encrypting plaintext and decrypting ciphertext (or keys between which a simple transformation exists).

|                    |                                            |
|--------------------|--------------------------------------------|
|                    |                                            |
| <b>Advantages</b>  | Fast and scalable                          |
| <b>Limitations</b> | Secure key exchange is a genuine challenge |

### 7.1 Stream Ciphers vs. Block Ciphers

- **Stream cipher** — processes input elements (e.g., one plaintext byte) continuously, producing one output element at a time as it goes. Commonly implemented via bitwise **XOR** of the plaintext with a key stream.

Plain (binary): 0101110Key (XOR with): 1001010Cipher: 1100100

- **Block cipher** — processes input a block of elements at a time, producing one output block per input block. E.g., encrypting "FOUR-AND-FOUR" as three separate blocks (FOUR, AND, FOUR), each translated back independently during decryption.

## 8. Classical Encryption Techniques

All classical encryption techniques rest on two basic building blocks:

**substitution** (each plaintext element is mapped to another element) and **transposition** (plaintext elements are rearranged).

### 8.1 Substitution Techniques

A substitution technique replaces plaintext letters with other letters, numbers, or symbols (or, at the bit level, replaces plaintext bit patterns with ciphertext bit patterns).

**Substitution techniques covered across the source material:**

1. Caesar Cipher
2. Modified version of the Caesar Cipher
3. Monoalphabetic Cipher
4. Homophonic Substitution Cipher
5. Polygram Substitution Cipher
6. Polyalphabetic Substitution Cipher (e.g., Vigenère)
7. Playfair Cipher
8. Hill Cipher

#### Caesar Cipher (Shift Cipher)

The earliest known substitution cipher, used by Julius Caesar: each letter of the alphabet is replaced by the letter three places further down (wrapping around, so the letter following 'z' is 'a').

Plaintext: pay more money  
Ciphertext: SDB PRUH PRQHB

For plaintext letter  $p$ , ciphertext letter  $c = E(p) = (p + 3) \bmod 26$ .

Generalized to any shift  $k$ :

Encryption:  $C = E(p) = (p + k) \bmod 26$   
Decryption:  $P = D(C) = (C - k) \bmod 26$  where  $1 \leq k \leq 25$

## Playfair Cipher

The best-known multi-letter (digraph) substitution cipher — treats pairs of plaintext letters as single units, translated into ciphertext digraphs. Built on a  $5 \times 5$  matrix of letters constructed from a keyword (e.g., "monarchy"), filled left-to-right/top-to-bottom with the keyword's letters (minus duplicates), then the remaining alphabet letters in order. **'I' and 'J' occupy the same cell.**

```
M O N A R  
C H Y B D  
E F G I/J K  
L P Q S T  
U V W X Z
```

**Encryption rules** (plaintext encrypted two letters — a digraph — at a time):

- A repeating pair falling in the same position is separated with a filler letter (e.g., 'x').
- Letters in the **same row** → each replaced by the letter to its right (wrapping to the row's start).
- Letters in the **same column** → each replaced by the letter beneath it (wrapping to the column's top).
- Otherwise → each letter is replaced by the letter in its own row **and** the column of the other letter (forming a rectangle).

Example: meet me at the school house → digraphs me et me at th es ch ox ol ho us ex  
→ ciphertext CL KL CL RS PD IL HY AV MP HF XL IU.



**Strength:** Since  $26 \times 26 = 676$  possible digraphs exist, identifying individual digraphs (and thus recovering frequency statistics) is far harder than for a simple monoalphabetic cipher.

## Polyalphabetic Ciphers — Vigenère Cipher

Improves on simple monoalphabetic substitution by using **different** monoalphabetic substitution rules as encryption proceeds through the message. All such techniques share: a set of related monoalphabetic substitution rules, and a key that determines which specific rule applies at each point.

The Vigenère scheme uses 26 Caesar ciphers (shifts 0–25), each denoted by a key letter (e.g., shift-3 = key letter 'd', since  $a=0$ ,  $b=1$ ,  $c=2$ ...). A **Vigenère tableau** lays out all 26 ciphers, keyed by row, with the plaintext alphabet across the top. To encrypt, a repeating keyword supplies a key letter per plaintext letter; the ciphertext letter sits at the intersection of the key-letter row and plaintext-letter column.

```
Key:      deceptive
Plaintext: wearediscoveredsaveyourself
Ciphertext: ZICVTWQNGRZGVTWAVZHCQYGLMGJ
```

**Strength:** multiple ciphertext letters can correspond to a single plaintext letter, and letter-frequency information is obscured — a substantial advance over a fixed monoalphabetic substitution.

## One-Time Pad Cipher

An **unbreakable** cryptosystem (against ciphertext-only attack) when used correctly. The message is represented as a sequence of 0s and 1s (via binary numbers or ASCII). The key is a truly **random** sequence of 0s and 1s of the **same length** as the message. Once used, a key is discarded and never reused.

```
Ci = Pi ⊕ Ki      (ciphertext bit = plaintext bit XOR key bit)
Pi = Ci ⊕ Ki      (decryption: identical XOR operation, same key)

Plaintext: 0 0 1 0 1 0 0 1
Key:       1 0 1 0 1 1 0 0
Cipher:    1 0 0 0 0 1 0 1
```

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      |                                                                                                                                                                                      |
| <b>Advantage</b>     | Completely unbreakable for a ciphertext-only attack                                                                                                                                  |
| <b>Disadvantages</b> | Requires a key as long as the message — expensive to produce and transmit; reusing a key for a second message is dangerous, since any knowledge of one message compromises the other |

## 8.2 Transposition Techniques

Rather than substituting symbols, transposition performs a **permutation** on the plaintext letters — rearranging their order without changing the letters themselves.

### Rail Fence

The simplest transposition cipher: plaintext is written diagonally across a set number of "rails," then read off row by row.

```
Plaintext (depth 2): meet at the school house
Row 1: m e a t e c o l o s
Row 2: e t t h s h o h u e
Ciphertext: MEATECOLOSETTHSHOHUE
```

### Row (Columnar) Transposition Cipher

A more complex scheme: the message is written into a rectangle row by row, then read off **column by column**, but the column order is permuted according to a numeric key — the order of columns becomes the algorithm's key.

**Weakness:** a *pure* transposition cipher is easily recognized because it preserves the same letter frequencies as the original plaintext. Security can be substantially improved by applying more than one stage of transposition, producing a more complex, less easily reconstructed permutation.

Other named techniques referenced: **Vernam Cipher** and **Book Cipher / Running Key Cipher**.

## 9. Advanced Encryption Standard (AES)

- A specification for the encryption of electronic data, established by the U.S. **NIST in 2001**.

- An encryption algorithm for securing sensitive material.
- Fixed **block size of 128 bits**, with a **key size of 128, 192, or 256 bits**.
- AES does **not** use a Feistel network.
- Main features: symmetric and parallel structure, well adapted to modern processors.

## 9.1 Symmetric Algorithms Compared

| Algorithm             | Strength          | Typical Use Case           |
|-----------------------|-------------------|----------------------------|
| AES (128/192/256-bit) | Highly secure     | VPNs, disk encryption, TLS |
| 3DES                  | Medium (outdated) | Legacy environments        |
| Blowfish / Twofish    | Secure            | File encryption tools      |

**Example flow:** Plaintext → Encrypt using shared key → Ciphertext → Decrypt using same key → Plaintext

## 10. Public-Key (Asymmetric) Cryptography

Traditional secret-key cryptography relies on a single key shared by both sender and receiver — if that key is disclosed, communications are compromised, and it does not protect the sender from a receiver forging a message and falsely attributing it (the two parties hold equal power over the shared secret).

**Public-key (two-key) cryptography** uses a *pair* of keys:

- A **public key** — may be known by anybody; used to **encrypt messages** and **verify signatures**.
- A **private key** — known only to the recipient; used to **decrypt messages** and **create (sign) digital signatures**.

Key properties:

- The public key is easily computed from the private key and other public information about the cipher (a polynomial-time / "P-time" problem).
- Knowing the public key and the cipher's public description, it remains computationally infeasible to derive the private key (an "NP-time" problem).
- The public key can thus be freely distributed to anyone wishing to communicate securely with its owner — though secure *distribution* of the

public key is itself a non-trivial problem (the **key distribution problem**), commonly addressed via digital certificates and a PKI (Section 12).

## 10.1 Three Important Classes of Public-Key Algorithms

- **Public-Key Distribution Schemes (PKDS)** — securely exchange a single piece of information (its value depends on both parties but can't be independently set); normally used as a session key for a subsequent private-key scheme.
- **Signature Schemes** — used solely to create a digital signature: the private key signs, the public key verifies.
- **Public Key Schemes (PKS)** — used for encryption: the public key encrypts, the private key decrypts.

Any public-key scheme can be used as a PKDS by simply selecting a message that *is* the required session key. Many public-key schemes are also signature schemes (provided encryption and decryption can be performed in either order).

## 10.2 Common Asymmetric Algorithms

| Algorithm                         | Purpose                                                     |
|-----------------------------------|-------------------------------------------------------------|
| RSA                               | Encryption and digital signatures                           |
| ECC (Elliptic Curve Cryptography) | Strong security with short key sizes                        |
| Diffie-Hellman                    | Secure key exchange                                         |
| DSA, ElGamal                      | Signature / encryption schemes based on discrete logarithms |

**Example flow:** Sender encrypts with the receiver's public key → receiver decrypts with their own private key.

| <b>Advantages</b>  | Secure key exchange, scalable                    |
|--------------------|--------------------------------------------------|
| <b>Limitations</b> | Computationally slower than symmetric encryption |

## 10.3 RSA Public-Key Cryptosystem

Proposed by **Rivest, Shamir & Adleman in 1977** (published 1978) — the best-known and most widely regarded practical public-key scheme. Used for

encrypting messages, exchanging keys, and creating digital signatures. Based on **exponentiation in a finite (Galois) field over integers modulo a prime**; its security rests on the difficulty of **factoring large numbers**.

**Key generation** (per user):

1. Select two large primes at random (~100 digits),  $p$  and  $q$ .
2. Compute the modulus  $R = p \times q$ .
3. Select an encryption key  $e$  at random such that  $e < R$  and  $\gcd(e, \phi(R)) = 1$ .
4. Solve the congruence  $e \cdot d \equiv 1 \pmod{\phi(R)}$  for the decryption key  $d$ , where  $0 \leq d \leq R$ .
5. Publish the **public key**:  $K1 = \{e, R\}$ .
6. Keep the **private key** secret:  $K2 = \{d, p, q\}$ .

Encryption:  $C = M^e \bmod R$   
 Decryption:  $M = C^d \bmod R$

This works because, for  $R = pq$  with distinct large primes  $p, q$ , and  $\phi(R) = (p-1)(q-1)$ :  $x^{\phi(R)} \equiv 1 \pmod R$  for all  $x$  not divisible by  $p$  or  $q$ .

**Worked example:** with  $e = 3$ ,  $R = 11 \times 47 = 517$ ,  $\phi(R) = 10 \times 46 = 460$ , solving the congruence gives  $d = 307$ . Encrypting  $M = 26$ :  $C = 26^3 \bmod 517 = 515$ . Decrypting:  $M = 515^{307} \bmod 517 = 26$ .

**Security of RSA** rests entirely on the difficulty of factoring the modulus  $R$ . As  $R$ 's bit-length grows, the number of bit-operations required by the best known factorization algorithms grows dramatically (from ~7,200 operations for a 20-digit  $R$  to  $\sim 2.36 \times 10^{19}$  operations for a 200-digit  $R$ ), which is why keys of several hundred bits are used in practice. Since ordinary computers can't natively handle numbers this large, RSA implementations depend on **multiple-precision arithmetic libraries**, and performance can be improved via techniques such as faster modulo reduction (e.g., Chivers' method), alternate multiplication algorithms (e.g., Schönhage–Strassen), and — for decryption specifically — the **Chinese Remainder Theorem**, which lets the (much smaller) computations be carried out modulo  $p$  and  $q$  separately rather than modulo the full  $R$ .

Note:  $e = 3$  is a commonly suggested small public exponent (usable across many users given appropriate precautions). RSA's algorithm was historically

patented in North America, which was a source of legal friction in early adoption.

## 10.4 Other Public-Key Schemes

- **ElGamal** — a variant of Diffie-Hellman key distribution enabling secure message exchange (published 1985); security rests on the difficulty of computing discrete logarithms. Involves key generation over a large prime  $p$  and a primitive root  $\alpha$ , with encryption producing a ciphertext pair  $\{C1, C2\}$  and decryption recovering the message via the recipient's private exponent.
- **Knapsack-based schemes** and **McEliece's Error-Correcting-Code-based schemes** — historically proposed alternatives; **all have since been broken**.
- The only public-key schemes still considered secure today are those based on exponentiation (RSA, Diffie-Hellman/ElGamal-style discrete-log schemes) — developing genuinely secure public-key schemes has proven difficult, which partly explains why adoption did not proceed as fast as their theoretical advantages might have suggested.

## 11. Hashing

Hashing converts data of any size to a **fixed-length, irreversible** value. Hashes cannot be decrypted and are used primarily for **verification**, not confidentiality.

| Algorithm                | Security Status                          |
|--------------------------|------------------------------------------|
| SHA-256 / SHA-512        | Secure and widely adopted                |
| MD5 / SHA-1              | Deprecated and vulnerable                |
| Argon2 / Bcrypt / PBKDF2 | Secure specifically for password storage |

**Use cases:** password storage, file integrity validation, blockchain transactions.

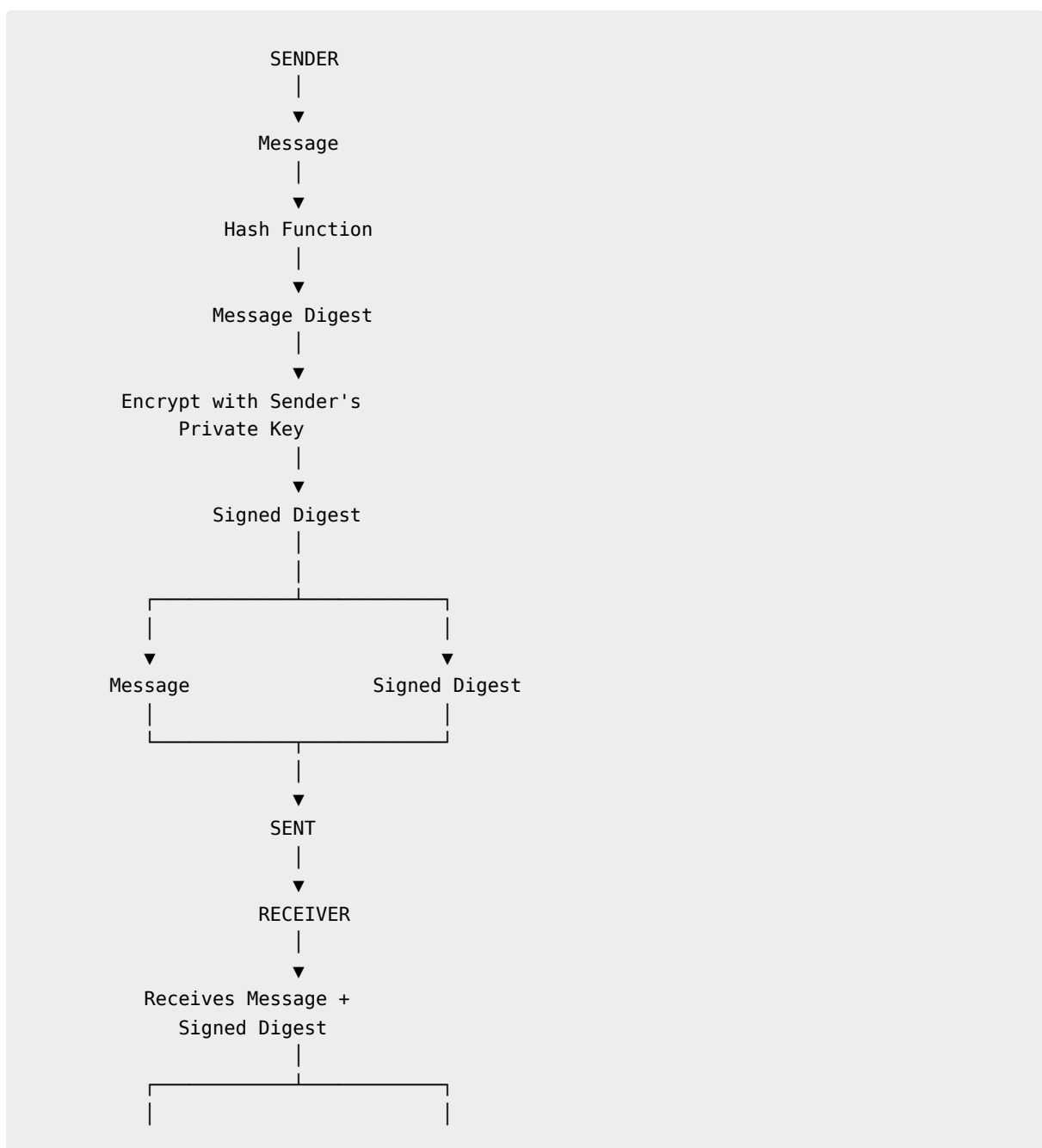
## 12. Digital Signatures

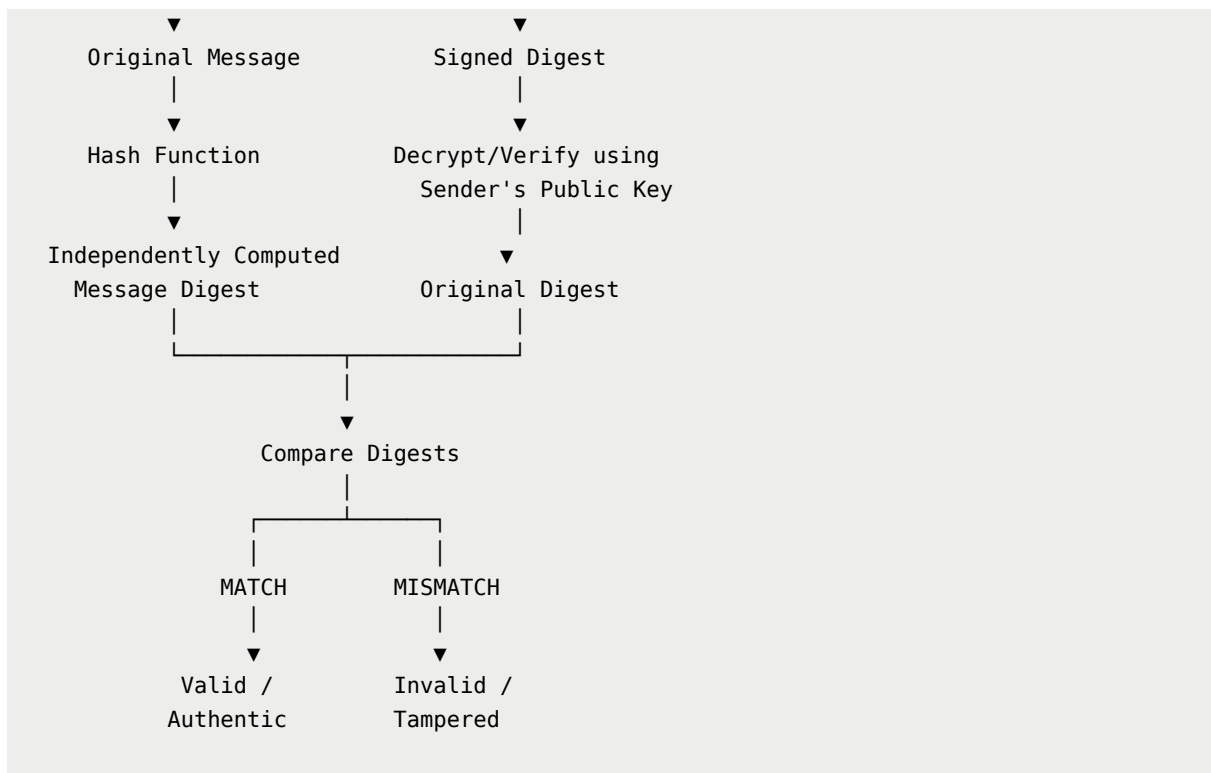
A digital signature is a mechanism authenticating a message — proving it effectively came from a given sender, much like a handwritten signature on a paper document. Digital signatures employ asymmetric cryptography. A valid digital signature gives the recipient reason to believe the message was created by a known sender who cannot deny sending it (**authentication and non-repudiation**), and that the message was not altered in transit (**integrity**).

Commonly used for software distribution, financial transactions, e-signatures, and legal compliance.

**Process:**

1. The data is hashed (producing a message digest).
2. The hash (digest) is encrypted using the **sender's private key**, producing the *signed digest*.
3. The receiver decrypts the signed digest using the **sender's public key** and compares it against an independently computed hash of the received message to validate it.





## 13. Digital Certificates and Public Key Infrastructure (PKI)

### 13.1 Digital Certificate

A public key certificate — also called a digital certificate or identity certificate — is an **electronic document used to prove the ownership of a public key**. In email encryption, code signing, and e-signature systems, a certificate's subject is typically a person or organization.

**A digital certificate typically includes:**

- Name of the subject
- Subject's public key
- Certification Authority's (CA) name
- Serial number
- Public Key Infrastructure (PKI) related metadata

### 13.2 What Is PKI?

PKI is the procedure of combining public keys with users' identities through the involvement of a **Certificate Authority (CA)**. This process can be carried out by



CA-side software or under direct human oversight. PKI is, broadly, an arrangement for **generating, storing, and sharing digital certificates** — certificates that are used to identify who a given public key belongs to. A **Registration Authority (RA)** identifies the user requesting information, working in concert with the CA.

**PKI flow (participants and roles):**

- **Sender** — holds a certificate private key and submits a request to the Registration Authority.
  - **Registration Authority** — validates the request and forwards it to the Certification Authority.
  - **Certification Authority** — issues the certificate (containing the certificate public key) back through the chain.
  - **Verification Authority** — validates signatures against certificates on behalf of the recipient.
  - **Recipient** — receives signed data and, via the Verification Authority, confirms the signature is valid.
- 

## 14. User Authentication

User authentication is a method that keeps unauthorized users from accessing sensitive information — e.g., User A has access only to relevant information and cannot see User B's sensitive data. Cybercriminals can gain access to a system and steal information when user authentication is not secure.

**User authentication can be provided using:**

- User IDs
  - Passwords
- 

## 15. Kerberos

Kerberos is a **computer network authentication protocol**. It provides identity assurance for users on a non-secure network, with information provided to both communicating parties. It runs on **port 88** by default, is based on **symmetric-key encryption**, and requires a **trusted third party**.

### 15.1 How Kerberos Works

Based on a **ticketing system**:

1. When a user logs in to the network, a session key is generated based on the user's password.
2. The session key request is forwarded to the **TGS (Ticket Granting Server)**.
3. The TGS provides a ticket to the user, which the user then forwards to the desired target server for authentication.
4. The ticket carries a **timestamp** issued by the TGS, so it can be used to access various services only within a given time window.
5. Timestamping keeps the ticket secure — it cannot be reused indefinitely by an unauthorized party once expired.

## 15.2 Kerberos Access Flow

1. User → Authentication Server (AS): sends a request
2. AS → User: issues a Ticket Granting Ticket (TGT)
3. User → Ticket Granting Server (TGS): sends a request (with the TGT)
4. TGS → User: issues a Service Ticket
5. User → Service Server: contacts it, presenting the Service Ticket
6. Service Server → User: grants access

## 16. Post-Quantum Cryptography

Emerging cryptographic methods designed specifically to resist attacks from quantum computers, which threaten to break the mathematical hardness assumptions (factoring, discrete logarithms) underlying RSA, Diffie-Hellman, and ECC.

**Examples:**

- CRYSTALS-Kyber
- Dilithium

## 17. Real-World Cybersecurity Use Cases

| Sector            | Cryptographic Application |
|-------------------|---------------------------|
| Web Security      | HTTPS, TLS, SSL           |
| Wireless Security | WPA3 (AES)                |

| Sector          | Cryptographic Application  |
|-----------------|----------------------------|
| Authentication  | PKI, MFA, secure tokens    |
| Data Protection | BitLocker, VeraCrypt       |
| Blockchain      | Hashing, ECC-based wallets |
| Email Security  | PGP, S/MIME                |

## 18. Cryptographic Attacks — Summary Table

| Attack            | Description                                   | Mitigation                         |
|-------------------|-----------------------------------------------|------------------------------------|
| Brute Force       | Repeated key guessing                         | Large key sizes, rate limiting     |
| Man-in-the-Middle | Intercepting encrypted data in transit        | Certificate pinning, TLS           |
| Cryptanalysis     | Exploiting weaknesses in the algorithm itself | Use modern, well-vetted algorithms |
| Hash Collision    | Two distinct inputs produce the same hash     | Avoid MD5/SHA-1; use SHA-256+      |

## 19. Tools Used in Cybersecurity for Cryptography

| Tool            | Category                        | Use Case                                |
|-----------------|---------------------------------|-----------------------------------------|
| OpenSSL         | Cryptographic framework         | Testing SSL/TLS, certificate management |
| GPG / PGP       | File and email encryption       | Secure communication                    |
| Hashcat         | Hash cracking                   | Password security testing               |
| John the Ripper | Password cracking               | Brute-force and dictionary attacks      |
| VeraCrypt       | Disk and container encryption   | Secure storage                          |
| CyberChef       | Browser-based cryptanalysis lab | Encoding, decoding, hashing             |
| Burp Suite      | Pentesting tool                 | Testing encryption in web applications  |

## 20. Best Practices for Strong Cryptographic Security

1. Use modern, secure algorithms — AES-256, SHA-256, ECC.

2. Avoid outdated/broken algorithms — DES, RC4, MD5, SHA-1.
  3. Apply key rotation policies and promptly revoke compromised keys.
  4. Implement proper key management using secure vaults or Hardware Security Modules (HSM).
  5. Use salting and key stretching for password storage (e.g., bcrypt, Argon2, PBKDF2).
  6. Enforce TLS 1.2 or higher for all encrypted network communications.
  7. Minimize cryptographic performance overhead by selecting an algorithm appropriate to the specific use case (e.g., symmetric for bulk data, asymmetric for key exchange/signing).
- 

## 21. Conclusion

Cryptography remains a critical foundation of cybersecurity. It enables secure communication, protects sensitive data, and supports authentication and integrity mechanisms across modern digital systems — from classical substitution and transposition ciphers through symmetric standards like AES, asymmetric systems like RSA, hashing, digital signatures, PKI, and authentication protocols like Kerberos. As cyber threats and computational power evolve (including the eventual arrival of practical quantum computing), professionals must stay current on cryptographic standards, known vulnerabilities, and emerging advancements such as post-quantum, quantum-safe encryption. Cryptography is not merely a tool but a discipline that plays a significant role in shaping secure digital architecture.